🛠️ Documento Técnico de Código Fuente

Sistema de Gestión de Proyectos - Assessment SENA

---

1. Organización del Proyecto

El proyecto está organizado siguiendo el patrón de **Arquitectura Limpia (Clean Architecture)** implementado completamente en el ecosistema **.NET 8** (Monolito Server-Side Rendering).

Estructura de Carpetas:

- **`CoursePlatform.Domain`:** Entidades base (`Project` , `TaskItem` , `ProjectEnrollment`), enums (`TaskProgressStatus`) y abstracciones libres de dependencias tecnológicas.
- **`CoursePlatform.Application`:** Servicios de capa lógica (`ProjectService.cs` , `TaskService.cs`), Interfaces, e Intercambio de Objetos DTO.
- **`CoursePlatform.Infrastructure`:** Implementación nativa de acceso a datos a través del `AppDbContext` de EF Core (PostgreSQL) y herramientas de seguridad (Identity).
- **`CoursePlatform.Api`:** Controladores Web MVC (`Controllers/Web/`), vistas Razor englobadas en HTML5 (`Views/`) y configuración central `Program.cs` .

2. Módulos y Flujo del Sistema

Este sistema abandona el uso de un cliente SPA (Single Page Applications) desvinculado, prefiriendo la solidez y SEO del enrutamiento MVC clásico donde el flujo es interactivo en el servidor (Render) -> Navegador.

Tecnologías y Frameworks:

- **Backend & Frontend Engine:** C# .NET 8, ASP.NET Core MVC (Razor Views renderizados del lado del servidor).
- **Estilos y UI:** CSS Vanilla (Custom Properties, Glassmorphism UI) definido globalmente en `site.css` .
- **Persistencia de Seguridad:** `Microsoft.AspNetCore.Identity.EntityFrameworkCore` gestionando encriptación y autologueo por `Cookies` .
- **Persistencia de Negocio:** PostgreSQL a través del ORM Entity Framework Core.

3. Explicación de Clases y Enrutadores Principales

`TasksController.cs` (Capa Web Api - MVC)

- Usa Autenticación restrictiva mediante atributos `[Authorize(Roles = "Admin")]` para alta gestión y `[Authorize]` estándar para interacción (cambios de estado).
- Coordina el renderizado de la vista de progreso de las tareas, incluyendo listados condicionales según si el usuario está inscrito (`isEnrolled`).

`ProjectService.cs` (Capa Application)

- **Regla de Negocio - Inscripciones:** Define el método `EnrollUserAsync` que guarda el `ProjectEnrollment` verificado dentro de una transición segura en la BD, evitando multi-enrollment accidental.
- **Regla de Negocio - Resumen Activo:** Envía al cliente el contexto dinámico de `Active Tasks` , totalizando las subtareas en estado `Completed` .

`AppDbContext.cs` (Capa Infrastructure)

- Establece a través del `FluentAPI` (`OnModelCreating`) los índices únicos compuestos, por ejemplo: `builder.Entity<ProjectEnrollment>().HasIndex(e => new { e.ProjectId, e.UserId`

```
}).IsUnique() .
```

4. Fragmentos de Código Relevantes

Lógica de Cambio de Estado (Trello/Kanban Flow)

```
public async Task<TaskItemDto> ChangeTaskStatus(Guid id, TaskProgressStatus newStatus)
{
    var task = await context.Tasks
        .FirstOrDefaultAsync(t => t.Id == id) ?? throw new Exception("Tarea no encontrada");

    // Lógica natural que permite que un Tarea en cualquier estado sea promovida a
Completada, Por Hacer o En Progreso.
    task.Status = newStatus;
    task.UpdatedAt = DateTime.UtcNow;

    await context.SaveChangesAsync();
    return ToDto(task);
}
```

Renderizado Inteligente Condicional en Razor

```
{{-- Status change form dentro de Index.cshtml --}}
<form action="/Tasks/ChangeStatus/@projectId/@t.Id" method="post">
    <select name="status" onchange="this.form.submit()">
        <option value="Todo" selected="@(t.Status == TaskProgressStatus.Todo ? "selected" :
null)">o Por Hacer</option>
        <option value="InProgress" selected="@(t.Status == TaskProgressStatus.InProgress ?
"selected" : null)">⏳ En Proceso</option>
        <option value="Completed" selected="@(t.Status == TaskProgressStatus.Completed ?
"selected" : null)">✓ Completada</option>
    </select>
</form>
```

5. Buenas Prácticas Aplicadas

- **Zero JavaScript AJAX:** Se emplea control nativo HTML `<form>` interceptando directivas básicas que el servidor consume, logrando una experiencia rápida y sin dependencias JavaScript.
- **Seguridad Activa (XSS):** Razor desinfecta (sanitize) nativamente todas las salidas del HTML renderizado limitando vulnerabilidades cruzadas.
- **Manejo de Transacciones Lógicas:** Soft Delete con Query Filters automáticos en Entity Framework para evitar exponer variables físicas en queries.